# DOCUMENTAÇÃO TÉCNICA

Engenharia reversa do sistema web da Oficina Mecânica AutoCerta

| | |
|---|---|
| **PROJETO** | Sistema web de gestão de oficina mecânica |
| **OBJETIVO** | Documentar requisitos, dados, API, interface, segurança e testes. |
| **IDENTIDADE VISUAL** | Preto e amarelo |
| **DATA** | 19/10/2026 |

*Resultados dos testes registrados após execução local: 23 aprovados e 1 reprovado (CT12-B).*

# Documentação Técnica — Oficina Mecânica AutoCerta

**Projeto:** Engenharia Reversa de Sistema Web
**Versão analisada:** arquivos recebidos em sistema.zip
**Data da documentação:** 19/10/2026
**Tecnologias identificadas:** Node.js (22.13.0 ou superior), Express, express-session, bcryptjs e SQLite (node:sqlite).

> **Nota de evidência:** este documento foi elaborado a partir dos arquivos-fonte enviados. Os resultados dos casos de teste foram registrados após a execução local do sistema. Os caminhos de arquivos são referências para confirmar detalhes no código.

## 1. Visão geral

O AutoCerta é uma aplicação web para apoiar uma oficina mecânica no gerenciamento de clientes, veículos e agendamentos de serviços. O navegador acessa páginas HTML estáticas servidas pelo Express. Os scripts JavaScript fazem requisições HTTP para a API, o servidor verifica a sessão, valida os dados e consulta ou altera o banco SQLite.

### 1.1 Como executar

**Requisitos:** Node.js 22.13 ou superior. No terminal do VS Code, entre na pasta `sistema` e execute:

```
node -v
npm install
npm start
```

Acesse `http://localhost:3000`. As credenciais de teste estão em `LEIA-ME.txt`. O banco `db/oficina_db.sqlite` é criado na primeira execução, usando `db/oficina_db.sql`. Apagar o arquivo SQLite e iniciar novamente recria o banco e apaga alterações feitas localmente; faça isso somente se desejar reiniciar os dados.

### 1.2 Estrutura relevante

| Arquivo/pasta | Responsabilidade observada |
|---|---|
| `server.js` | Inicializa Express, sessão, middleware de autenticação, rotas da API e servidor HTTP. |
| `seguranca.js` | Criptografa e descriptografa CPF usando AES-256-GCM. |
| `db/conexao.js` | Abre o SQLite e executa o script SQL quando o arquivo do banco ainda não existe. |
| `db/oficina_db.sql` | Define tabelas, chaves e dados iniciais. |
| `public/*.html` | Estrutura das telas do sistema. |
| `public/js/comum.js` | Funções compartilhadas para chamadas à API, sessão, escape HTML, formatação e modais. |
| `public/js/login.js` | Envia o formulário de login. |
| `public/js/clientes.js` | Lista, busca, cadastra e edita clientes. |
| `public/js/veiculos.js` | Lista e cadastra veículos associados a clientes. |
| `public/js/agendamentos.js` | Lista, cadastra e altera a data de agendamentos. |
| `public/css/style.css` | Estilos das páginas. |
| `package.json` | Dependências, comando de inicialização e versão mínima do Node.js. |

## 2. Requisitos funcionais

Os requisitos abaixo foram derivados das rotas e scripts presentes no projeto. A confirmação visual deve ser feita ao executar o sistema.

| ID | Requisito funcional | Evidência principal |
|---|---|---|
| **RF01** | Permitir autenticação com login e senha. | server.js — POST /api/login; public/js/login.js |
| **RF02** | Manter uma sessão de usuário após login válido. | server.js — configuração de express-session e req.session.usuario |
| **RF03** | Informar os dados do usuário autenticado. | GET /api/me; public/js/comum.js |
| **RF04** | Encerrar a sessão por meio da ação Sair. | POST /api/logout; public/js/comum.js |
| **RF05** | Listar clientes em ordem alfabética. | GET /api/clientes |
| **RF06** | Buscar clientes por nome, e-mail, telefone ou CPF. | GET /api/clientes?busca=... |
| **RF07** | Cadastrar cliente com nome e CPF e aceitar telefone/e-mail opcionais. | POST /api/clientes |
| **RF08** | Editar os dados de um cliente existente. | PUT /api/clientes/:id |
| **RF09** | Listar veículos junto ao nome do cliente relacionado. | GET /api/veiculos |
| **RF10** | Cadastrar veículo associado a um cliente existente. | POST /api/veiculos |
| **RF11** | Listar agendamentos com informações do veículo e do cliente. | GET /api/agendamentos |
| **RF12** | Cadastrar agendamento para veículo existente. | POST /api/agendamentos |
| **RF13** | Alterar a data de um agendamento existente. | PUT /api/agendamentos/:id |
| **RF14** | Agrupar visualmente os agendamentos por mês na tela. | public/js/agendamentos.js |
| **RF15** | Redirecionar para o login quando uma chamada protegida recebe HTTP 401. | public/js/comum.js |

## 7.3 Criptografia do CPF

`seguranca.js` usa o módulo `crypto` do Node.js e o algoritmo AES-256-GCM. A função gera um IV aleatório de 12 bytes, criptografa o CPF, obtém a tag de autenticação e armazena os componentes em Base64 separados por `:`. A descriptografia recupera os componentes e valida a tag. Como a criptografia é reversível, a aplicação consegue mostrar e pesquisar o CPF após descriptografá-lo.

## 7.4 Riscos e melhorias recomendadas

1. **Mover segredos para variáveis de ambiente:** o segredo de sessão e a frase usada para derivar a chave do CPF estão fixos no código. Em ambiente real, usar segredos fortes fora do repositório e documentar a rotação das chaves.
2. **Fortalecer cookies e transporte:** configurar sameSite e, em produção com HTTPS, secure: true; configurar proxy confiável corretamente se houver proxy reverso. httpOnly já está habilitado.
3. **Limitar tentativas de login:** adicionar rate limiting e atrasos/monitoramento para dificultar tentativas automatizadas de senha.
4. **Validar CPF de forma completa:** além de contar 11 dígitos, validar dígitos verificadores e manter unicidade garantida também no banco. Como o CPF é criptografado com IV aleatório, a restrição UNIQUE direta no texto criptografado não impediria duplicatas lógicas.
5. **Reforçar validações de entrada:** validar tipos e intervalos dos IDs e tratar exceções de banco com respostas consistentes, sem expor detalhes internos.
6. **Rever permissões:** todos os usuários autenticados parecem poder usar as mesmas rotas. Implementar autorização por perfil se cada função da oficina precisar de permissões diferentes.

# 8. Casos de teste

**Resumo da execução:** 24 casos de teste executados localmente com dados fictícios: **23 aprovados** e **1 reprovado (CT12-B — busca de clientes por CPF)**. A coluna “Resultado real” compara o que foi observado com o resultado esperado.

| ID | Req./área | Procedimento | Resultado esperado | Resultado real |
|---|---|---|---|---|
| **CT01** | RF01 Login válido | Abrir login e entrar com uma credencial listada em LEIA-ME.txt. | Redirecionar para principal.html. | **APROVADO** Conforme o esperado. |
| **CT02** | RF01 Login inválido | Informar login/senha incorretos. | Mensagem de credenciais inválidas; permanecer no login. | **APROVADO** Conforme o esperado. |
| **CT03** | RF01 Validação de campos | Enviar POST /api/login sem login ou sem senha. | HTTP 400 com mensagem de campos obrigatórios. | **APROVADO** Conforme o esperado. |
| **CT04** | RF03 Sessão | Após login, abrir /api/me. | HTTP 200 com id e nome do usuário. | **APROVADO** Conforme o esperado. |
| **CT05** | RF04 Logout | Clicar em Sair e depois consultar /api/me. | Sessão encerrada; consulta protegida deve responder 401. | **APROVADO** Conforme o esperado. |
| **CT06** | RF15 Proteção de rota | Sem login, chamar GET /api/clientes. | HTTP 401. | **APROVADO** Conforme o esperado. |
| **CT07** | RF15 Expiração | Deixar a sessão expirar e acionar uma operação protegida. | API responde 401 e front-end redireciona para login com aviso. | **APROVADO** Conforme o esperado. |
| **CT08** | RF07 Cliente válido | Cadastrar cliente com nome, CPF fictício de 11 dígitos e e-mail válido. | HTTP 201; cliente aparece na listagem. | **APROVADO** Conforme o esperado. |
| **CT09** | RF07 Nome obrigatório | Enviar cliente com nome vazio. | HTTP 400. | **APROVADO** Conforme o esperado. |
| **CT10** | RF07 CPF inválido | Enviar CPF com menos de 11 dígitos. | HTTP 400. | **APROVADO** Conforme o esperado. |
| **CT11** | RF07 CPF duplicado | Tentar cadastrar novamente o mesmo CPF fictício. | HTTP 400 informando CPF já cadastrado. | **APROVADO** Conforme o esperado. |
| **CT12** | RF06 Busca por nome | Buscar por parte do nome de um cliente existente. | A lista deve conter clientes correspondentes. | **APROVADO** Conforme o esperado. |
| **CT12-B** | RF06 Busca por CPF | Buscar pelo CPF fictício de um cliente existente. | A lista deve conter o cliente correspondente. | **REPROVADO** Buscar por CPF não retornou nenhum cliente (a lista veio vazia). |
| **CT13** | RF08 Edição de cliente | Editar nome ou telefone de cliente existente. | HTTP 200 e listagem atualizada. | **APROVADO** Conforme o esperado. |
| **CT14** | RF10 Placa inválida | Enviar veículo com placa fora do padrão. | HTTP 400. | **APROVADO** Conforme o esperado. |
| **CT15** | RF10 Cliente obrigatório | Enviar veículo sem cliente_id. | HTTP 400. | **APROVADO** Conforme o esperado. |
| **CT16** | RF10 Placa duplicada | Cadastrar veículo usando placa já existente. | HTTP 400 informando placa repetida. | **APROVADO** Conforme o esperado. |
| **CT17** | RF10 Veículo válido | Cadastrar veículo com placa válida e cliente existente. | HTTP 201 e veículo aparece na listagem. | **APROVADO** Conforme o esperado. |

| ID | Req./área | Procedimento | Resultado esperado | Resultado real |
|---|---|---|---|---|
| **CT18** | RF12 Data inválida | Enviar agendamento com data 2026-02-31. | HTTP 400. | **APROVADO** Conforme o esperado. |
| **CT19** | RF12 Serviço obrigatório | Enviar agendamento com serviço vazio. | HTTP 400. | **APROVADO** Conforme o esperado. |
| **CT20** | RF12 Agendamento válido | Criar agendamento com veículo existente, data real e serviço. | HTTP 201 e agendamento aparece na listagem. | **APROVADO** Conforme o esperado. |
| **CT21** | RF13 Edição de data | Alterar a data de agendamento existente para data válida. | HTTP 200; a data exibida muda. | **APROVADO** Conforme o esperado. |
| **CT22** | RF13 ID inexistente | Fazer PUT /api/agendamentos/999999 com data válida. | HTTP 404. | **APROVADO** Conforme o esperado. |
| **CT23** | Segurança/XSS | Cadastrar texto fictício contendo <script> em campo textual e abrir a listagem. | O texto deve aparecer escapado, sem executar script. | **APROVADO** Conforme o esperado. |

### 8.1 Como registrar os resultados reais

1. Inicie o sistema e anote o resultado de cada procedimento.
2. Abra as ferramentas do navegador com F12 e use a aba Network para registrar URL, método, status e resposta.
3. Salve capturas de tela em `docs/evidencias/`, por exemplo `CT01-login-valido.png`.
4. Para testes de API, use a aba Console, Postman ou cURL, mantendo uma sessão válida quando necessário.
5. Compare o resultado real ao esperado e registre Aprovado ou Reprovado, mantendo as evidências em `docs/evidencias/`.

## 9. Fluxograma de uma funcionalidade: cadastrar veículo

**Diagrama — código Mermaid**

```
flowchart TD
A[Usuário abre Veículos] --> B[veiculos.js solicita GET /api/veiculos]
B --> C{Sessão válida?}
C -- Não --> D[API retorna 401]
D --> E[comum.js redireciona ao login]
C -- Sim --> F[Servidor consulta veículos e clientes]
F --> G[Front-end mostra listagem]
G --> H[Usuário clica em Novo]
H --> I[GET /api/clientes para preencher seleção]
I --> J[Usuário preenche placa, modelo, tipo, marca e cliente]
J --> K[POST /api/veiculos]
K --> L{Validações passaram?}
L -- Não --> M[API retorna 400 com erro]
M --> N[Front-end mostra mensagem]
L -- Sim --> O[INSERT na tabela veiculos]
O --> P[API retorna 201]
P --> Q[Front-end mostra sucesso e recarrega a lista]
```

## 10. Observações sobre o código

- As consultas SQL usam parâmetros (?) para valores variáveis, reduzindo o risco de SQL injection por concatenação de entrada.

- `esc()` é importante porque algumas tabelas são montadas por strings atribuídas a `innerHTML`. A função codifica caracteres especiais antes de inserir valores dinâmicos.
- O JOIN na listagem de veículos permite exibir o nome do cliente associado; os JOINs da listagem de agendamentos recuperam informações relacionadas para evitar mostrar apenas IDs.
- A listagem de agendamentos é ordenada no SQL e agrupada visualmente no JavaScript. São responsabilidades diferentes.
- A base não contém rotas de exclusão nem edição de veículos; não documente essas ações como funcionalidades existentes.
- O banco SQLite só é inicializado a partir do SQL quando o arquivo `oficina_db.sqlite` ainda não existe.

## 11. Checklist antes de entregar

| | |
|---|---|
| [ ] | Confirmar requisitos e regras com leitura do código e execução. |
| [ ] | Conferir o DER com `db/oficina_db.sql`. |
| [ ] | Revisar todas as rotas de `server.js`. |
| [ ] | Preencher os resultados reais dos testes e salvar evidências. |
| [ ] | Exportar este documento para `docs/documentacao.pdf`. |
| [ ] | Exportar o DER como `docs/der.png`. |
| [ ] | Criar a cópia comentada do código em `sistema_comentado/`. |
| [ ] | Manter node_modules, banco local, .env e segredos fora do Git. |
| [ ] | Fazer commits pequenos em pelo menos três dias diferentes, conforme o enunciado. |